

Polynomial-Delay Enumeration of Fixed-Endpoint Vertex-Regular Paths in Skew-Symmetric Digraphs

Lennart Rudolph 

August 11, 2026

Abstract

Let $G = (V, E)$ be a finite skew-symmetric digraph with fixed-point-free involution $v \mapsto \bar{v}$. A directed path is vertex-regular if it contains no pair $v, \bar{v}$. For prescribed vertices s, t , we give a depth-first algorithm that enumerates every vertex-regular s - t path exactly once with delay $O(n^2(n + m))$ and polynomial space. Its extension test deletes the vertices of the current prefix and their complements and invokes linear-time regular reachability, obtained from the arc-regular algorithm of Goldberg and Karzanov by vertex splitting. This establishes the fixed-endpoint variant needed for the two-unit-clause consequence stated after Conjecture 8.1 of Kullmann and Clewer. Via their regular-path bijection, it also yields $O(\ell(F)^3)$ -delay enumeration of the minimally unsatisfiable subsets of a 2-CNF containing two prescribed distinct unit clauses. An executable implementation and a Lean companion checking the vertex-splitting model interface and key prefix-oracle lemmas accompany the note.

Keywords: enumeration algorithms; polynomial delay; skew-symmetric digraphs; regular paths; minimal unsatisfiable subsets; 2-CNF

1 Introduction and result

Kullmann and Clewer study minimally unsatisfiable subsets (MUSs) of 2-CNFs through regular paths in implication digraphs. Their Theorem 6.1 identifies MUSs containing two prescribed distinct unit clauses $\{x\}, \{y\}$ with vertex-regular paths from x to $\bar{y}$, and their Conjecture 8.1 asks for polynomial-delay enumeration of regular paths in digraphs with given skew-symmetry [5]. All references here are to version 1 of that preprint, submitted March 11, 2026.

Polynomial delay is a classical output-sensitive standard [4]; reverse search and backtracking give two broad enumeration paradigms [1, 7]. The present note does not improve those general paradigms. Its point is that the constrained prefix-extension question for vertex-regular paths is itself a regular- reachability instance. On the satisfiability side, general MUS computation has a substantial algorithmic literature [6]; our corollary is narrower and exploits the exact 2-CNF path bijection.

The endpoint convention matters. If “regular paths” means paths with arbitrary endpoints, a depth-first traversal can simply output every regular prefix; no endpoint-constrained extension oracle is needed. The substantive form implicit in the stated two-unit-clause consequence prescribes the two endpoints. We prove precisely that form.

A *skew-symmetric digraph* is a finite simple digraph $G = (V, E)$ with a fixed-point-free involution $v \mapsto \bar{v}$ such that $(u, v) \in E$ if and only if $(\bar{v}, \bar{u}) \in E$. The mate of an arc $e = (u, v)$ is $\bar{e} = (\bar{v}, \bar{u})$. A path is a nonempty directed simple vertex sequence; in particular, (t) is a path of length zero from t to t . A path is *vertex-regular* if it contains no pair $v, \bar{v}$. An arc-regular path contains no pair $e, \bar{e}$. The distinction is essential: Goldberg and Karzanov use the arc convention [3], whereas Kullmann and Clewer use the vertex convention [5].

Let $n = |V|$, $m = |E|$, and let $\Delta^+(G)$ be the maximum outdegree. Define $\text{RPath}(G, s, t)$ to mean that G has a vertex-regular s - t path, with $\text{RPath}(G, t, t) = \text{true}$ and with value false if an endpoint is absent.

Theorem 1. *For every prescribed $s, t \in V$, all vertex-regular s - t paths of G can be enumerated exactly once with delay*

$$O(n(\Delta^+(G) + 1)(n + m)) \subseteq O(n^2(n + m))$$

and polynomial space. Delay includes preprocessing, the time to the first output, the time between consecutive outputs, and termination after the last output.

Thus Theorem 1 establishes the fixed-endpoint variant needed for the two-unit-clause consequence stated after Conjecture 8.1. The flashlight, or extension-oracle, paradigm is standard in enumeration algorithms [2]. The contribution is the exact prefix-extension equivalence and its application here, not a new general self-reduction technique.

2 Fixed-endpoint regular reachability

The only external algorithmic input is Goldberg and Karzanov's regular-path algorithm [3, Sections 2.4–2.6 and Theorem 2.8]. Their r-reachability problem prescribes complementary vertices $a, \bar{a}$ in a finite skew-symmetric digraph, and an r-path may not contain both an arc and its mate. Their adjacency-list algorithm decides reachability in $O(M)$ time and extracts a path in an additional $O(N+M)$ time for an instance with N vertices and M arcs. We therefore use the uniform $O(N+M)$ bound. Kullmann and Clewer invoke this result for vertex-regular reachability in their Corollary 6.2 [5, Corollary 6.2]. The direct linear-time algorithm, not Goldberg and Karzanov's separate larger reduction to matching, supplies the asymptotic bound below; the executable companion uses that matching reduction only as a transparent polynomial-time reference implementation.

The following lemma records completely the linear-size conversion needed here. It also makes the endpoint gadget explicit; this is the place where an overline error would change the problem.

Lemma 2 (vertex-regular reachability). *RPath(G, s, t) can be decided in $O(n+m)$ time. For $s \neq t$ and $s \neq \bar{t}$, the decision reduces to arc-regular reachability between a fresh complementary pair in a skew-symmetric digraph with $2n+2$ vertices and at most $m+n+4$ arcs.*

Proof. The cases $s = t$ and $s = \bar{t}$ have answers true and false, respectively: the first is witnessed by the singleton path, while the endpoints in the second case already clash. Hence assume $s \neq t, \bar{t}$. First remove every *unit arc* $(v, \bar{v})$. Such an arc cannot occur in a vertex-regular path, and its mate is itself.

Split every vertex v into v^-, v^+ and set

$$\overline{v^-} = \bar{v}^+, \quad \overline{v^+} = \bar{v}^-.$$

Add the internal arc $i_v = (v^-, v^+)$, and replace every remaining arc (u, v) by $a_{uv} = (u^+, v^-)$. The mate of i_v is $i_{\bar{v}}$, and the mate of a_{uv} is $a_{\bar{v}\bar{u}}$; thus the split digraph is skew-symmetric. The split-node involution swaps the two polarities and has no fixed point. An internal arc is never fixed by the arc involution, while a transformed arc is fixed exactly when it came from a discarded unit arc $(u, \bar{u})$. Consequently every retained split arc lies in a genuine two-element mate pair.

A path $(v_0, \dots, v_k)$ lifts to the alternating path

$$v_0^-, v_0^+, v_1^-, v_1^+, \dots, v_k^-, v_k^+.$$

If the original path is vertex-regular, its lift cannot contain both $i_v, i_{\bar{v}}$. Nor can it contain both a_{uv} and $a_{\bar{v}\bar{u}}$, since that would put v and $\bar{v}$ on the original path. Hence the lift is arc-regular. Conversely, in the gadget-free split digraph every path from a minus copy to a plus copy alternates: the only arc leaving v^- is i_v , and every arc leaving v^+ is a transformed original arc. Contracting each consecutive pair v^-, v^+ therefore projects the path to a directed path in G . A repeated projected vertex would repeat its two split copies, so simplicity is preserved. Finally, arc-regularity forbids the mate pair $i_v, i_{\bar{v}}$ and hence forbids both v and $\bar{v}$ in the projection. This proves the two-way vertex-splitting equivalence for compatible endpoints.

It remains to make the endpoints complementary, as required by the standard arc-regular reachability instance. Add fresh vertices $a, \bar{a}$ and the four boundary arcs

$$e_s = (a, s^-), \quad \bar{e}_s = (\bar{s}^+, \bar{a}), \quad e_t = (t^+, \bar{a}), \quad \bar{e}_t = (a, \bar{t}^-).$$

Figure 1 displays the two possible nonforbidden boundary traversals.

Because $s \neq \bar{t}$, the four boundary arcs are distinct; they form exactly the two displayed mate pairs, so neither a fixed arc nor a parallel arc is introduced. The fresh vertex a has no incoming arc and only $e_s, \bar{e}_t$ leave it, while $\bar{a}$ has no outgoing arc and only $e_t, \bar{e}_s$ enter it. Thus every $a-\bar{a}$ path chooses exactly one first boundary arc and exactly one last boundary arc. The choices $(e_s, \bar{e}_s)$ and $(\bar{e}_t, e_t)$ each contain a mate pair and are therefore impossible for an arc-regular path. The choice (e_s, e_t) projects to an $s-t$ path. The remaining choice $(\bar{e}_t, \bar{e}_s)$ projects to a $\bar{t}-\bar{s}$ path, whose complemented reverse is an $s-t$ path. The preceding splitting argument proves regularity in both cases. Conversely, an $s-t$ vertex-regular path lifts between e_s and e_t ; the assumptions $s \neq \bar{t}$ and vertex-regularity

$$\begin{array}{ccc}
a \xrightarrow{e_s} s^- & \rightsquigarrow & t^+ \xrightarrow{e_t} \bar{a} \\
a \xrightarrow{\bar{e}_t} \bar{t}^- & \rightsquigarrow & \bar{s}^+ \xrightarrow{\bar{e}_s} \bar{a}
\end{array}$$

Figure 1: The fresh endpoint gadget. The mate pairs are $e_s \leftrightarrow \bar{e}_s$ and $e_t \leftrightarrow \bar{e}_t$. The upper traversal projects to an s - t path; the lower one projects to its complemented reverse.

exclude every boundary or internal mate pair. This exhausts all four first-last combinations. The construction is an existence reduction rather than a bijection: the two allowed traversals encode the same existence statement, since complementation and reversal take one projected path to the other.

After unit-arc removal the oracle instance has $N = 2n + 2$ vertices and at most $M = n + m + 4$ arcs. Its adjacency lists, mate pointers, and the projection of a returned witness are constructed in $O(n + m)$ time. The cited $O(N + M)$ arc-regular decision-and-extraction bound therefore gives $O(n + m)$ here. Loops require no case because the input digraph is simple; arbitrary prescribed endpoints are handled by the two direct cases and the fresh-pair construction above. $\square$

3 Polynomial-delay enumeration

For $X \subseteq V$, put $\bar{X} = \{\bar{x} : x \in X\}$. Let $P = (v_0, \dots, v_k)$ be a vertex-regular prefix with $v_0 = s$ and $v_k = v$. For an admissible successor

$$w \in N_G^+(v) \setminus (V(P) \cup \bar{V}(P)),$$

define the induced skew-symmetric digraph $H_P = G - (V(P) \cup \bar{V}(P))$.

Lemma 3 (prefix extension). *There is a vertex-regular s - t path with prefix $P; w$ if and only if $\text{RPath}(H_P, w, t)$ is true.*

Proof. Suppose a full path extends $P; w$. Delete the vertices of P from its vertex sequence. The remaining suffix begins at w , ends at t , remains vertex-regular, and contains neither an old prefix vertex nor the complement of one. It is therefore a path in H_P .

Conversely, let $Q = (w = q_0, q_1, \dots, q_r = t)$ be a vertex-regular path in H_P . Concatenate the sequence P , the arc $v \rightarrow w$, and the tail $(q_1, \dots, q_r)$ of Q ; equivalently, append Q while omitting its first vertex w . The two pieces are simple and regular. Every vertex of Q avoids $V(P) \cup \bar{V}(P)$, and regularity of Q excludes $\bar{w}$. Thus neither a repetition nor a complementary pair crosses the join. The concatenation is the required vertex-regular extension. $\square$

Fix any deterministic order of every adjacency list. The enumeration procedure is as follows; H_P is computed once per prefix, outside the candidate loop, and the convention $\text{RPath}(H, t, t) = \text{true}$ covers the last arc without a special case.

Procedure $\text{ENUMERATE}(P = (v_0, \dots, v_k))$

1. If $v_k = t$, output P and return.
2. Set $H_P \leftarrow G - (V(P) \cup \bar{V}(P))$.
3. For each $w \in N_G^+(v_k) \cap V(H_P)$, in the fixed order:
if $\text{RPath}(H_P, w, t)$, call $\text{ENUMERATE}(P; w)$.

The initial call is $\text{ENUMERATE}((s))$. When $s = t$ it outputs the singleton; when $s = \bar{t}$, every extension test is false because t is deleted. Unit arcs are never admissible, and loops are absent.

Proof of Theorem 1. By Lemma 3, a recursive child is entered exactly when its prefix has at least one completion. Induction on the number of remaining vertices shows that every viable prefix reaches an output, and that every vertex-regular s - t path follows its own successive prefixes and is output. Conversely, every output is regular by the same lemma. A vertex sequence has a unique chain of prefixes, so no two recursion branches output the same path. Each recursive step adds a new vertex and its mate remains forbidden; the depth is at most n , proving termination.

Let B denote a complete frame charge: constructing H_P , scanning the whole successor list, and obtaining every oracle answer encountered during that scan. Lemma 2 gives $B = O((\Delta^+(G) + 1)(n + m))$. Every nonroot frame

entered by the depth-first search is viable and therefore has an output descendant. If no output exists, exactness of the oracle prevents any recursive call, so the root finishes within one B charge. Otherwise the root is viable as well. Before the first output, at most n new frames are entered. Between two consecutive outputs, at most n old frames are resumed while the search retreats, and at most n new frames are entered on the next descent. Once a viable child is entered, it produces an output before control can return past that child, so no further old frame can be resumed in the same interval. After the final output, at most n saved scans remain; any positive answer would instead have an output descendant and contradict finality. Thus preprocessing, first-output, inter-output, and post-output delay are all $O(n(\Delta^+(G) + 1)(n + m))$. When $\Delta^+(G) \geq 1$, this simplifies to $O(n\Delta^+(G)(n + m))$. Uniformly, $\Delta^+(G) + 1 \leq n + 1$ gives the advertised $O(n^2(n + m))$ bound. Writing an output costs only $O(n)$.

The stack has depth at most n . Even the direct strategy of retaining one $O(n + m)$ induced graph, prefix, deletion mask, and adjacency iterator per frame, while reusing one adjacency-list oracle workspace, uses $O(n(n + m))$ space. $\square$

Remark 4 (arbitrary endpoints). *If all ordered endpoints are allowed, outputting every regular DFS prefix already gives a direct polynomial-delay enumeration. Alternatively one may run Theorem 1 over all ordered pairs. This easy variant is not a separate contribution and is why the theorem and title emphasize fixed endpoints.*

4 Two prescribed unit clauses

Let F be a clash-free 2-clause-set, in the sense of Kullmann and Clewer, containing no empty clause; let $\ell(F)$ denote its encoded length, and let $\{x\}, \{y\} \in F$ be distinct unit clauses. Kullmann and Clewer’s rendered Theorem 6.1—with the overline on the second endpoint—is the bijection

$$P \in \mathfrak{R}_{x,\bar{y}}(\text{idg}(F)) \mapsto \mathcal{CL}(E(P)) \cup \{\{x\}, \{y\}\} \in \text{MU}_{\{x\},\{y\}}(F).$$

Theorem 5 (two-unit-clause MUS enumeration). *All MUSs of F containing both prescribed unit clauses can be enumerated exactly once with $O(\ell(F)^3)$ delay and polynomial space.*

Proof. Constructing $\text{idg}(F)$ takes $O(\ell(F))$ time and yields $n, m = O(\ell(F))$. Apply Theorem 1 with the fixed endpoints $s = x$ and $t = \bar{y}$. Translating a path to its clause set takes at most $O(\ell(F))$ time using the implication-arc labels. Theorem 6.1 is a bijection, so completeness and absence of duplicates are preserved rather than recovered by postprocessing. Substituting $n, m = O(\ell(F))$ in the $O(n^2(n + m))$ bound gives $O(\ell(F)^3)$ delay; construction and output translation are absorbed. $\square$

Theorem 5 is exactly the consequence stated after Conjecture 8.1. It does not give polynomial delay for MUSs with only one specified unit clause, for all MUSs of an arbitrary 2-CNF, or for the harder structural families discussed by Kullmann and Clewer. It also makes no claim of practical superiority over specialized MUS enumerators.

5 Verification artifact

The ancillary artifact contains a dependency-free Python enumerator and oracle. The oracle implements vertex splitting, the Goldberg–Karzanov matching reduction, and an Edmonds blossom search. A separately written exponential specification is used only by the tests. The default suite compares every enumerated path and every oracle answer on 10,192 endpoint instances; the exhaustive-six mode covers all 4,096 effective six-vertex arc-orbit subsets and 157,648 endpoint instances. These finite tests support the implementation but do not prove the theorem. This transparent reference oracle is polynomial-time; it does not implement the cited linear-time oracle, so its measured running time is not evidence for the theorem’s sharper delay bound.

The Lean 4.32.2 file checks four interfaces used above: (i) the split-node and split-arc involutions, deletion of the only possible fixed transfer arcs, and skew-symmetry of the split edge relation; (ii) equivalence between vertex clash-freeness of a supported trace and arc clash-freeness of its lifted arc set, together with the exact alternating-walk lift; (iii) the bidirectional list-regularity join/deletion theorem and exact-length composition of the corresponding relational walks; and (iv) an abstract exact completion-oracle interface, including viability of every accepted prefix and uniqueness of a prefix at a fixed depth. CSLib v4.32.2 and mathlib v4.32.2 are pinned to immutable commits in the manifest, and the project contains no `sorry` or custom axiom.

This is deliberately not an end-to-end formalization. The fresh endpoint gadget, the published Goldberg–Karzanov algorithm, the Python matching oracle, the full recursive enumerator, and the asymptotic delay and

space analysis remain outside Lean. The abstract oracle carries its correctness as a hypothesis, and the formal graph witnesses are relational walks rather than machine-checked simple paths.

Data and code availability. The Python sources, independent specification, tests, Lean project, exact dependency manifest, build instructions, and checksum manifest are supplied in the accompanying reproducibility archive named

`regular-path-delay-ancillary-artifact.tar.gz`.

Its SHA-256 checksum is

77245ed05f542e91a8b2f6e101d347e89d1b28cca82f281c2152c3cb60871178

The paper source and PDF are submitted separately from this reproducibility archive. The frozen public release is available at <https://doi.org/10.5281/zenodo.21892986>.

Declaration of agentic AI and AI-assisted technologies. During the discovery, verification, and preparation of this work, the author used OpenAI Codex and Anthropic Claude as agentic research assistants. Codex assisted with literature and novelty searches, proof development, manuscript organization and editing, Python implementation and testing, and Lean formalization. Claude was used for independent adversarial review, including attempts to identify mathematical, computational, citation, and presentation defects. The author independently checked the claims, proofs, computations, sources, and final text, accepts full responsibility for the work, and confirms that neither system is an author.

References

- [1] David Avis and Komei Fukuda. Reverse search for enumeration. *Discrete Applied Mathematics*, 65(1–3):21–46, 1996. doi: 10.1016/0166-218X(95)00026-N. URL [https://doi.org/10.1016/0166-218X\(95\)00026-N](https://doi.org/10.1016/0166-218X(95)00026-N).
- [2] Florent Capelli and Yann Strozecki. Enumerating models of DNF faster: Breaking the dependency on the formula size. *Discrete Applied Mathematics*, 303:203–215, 2021. doi: 10.1016/j.dam.2020.02.014. URL <https://arxiv.org/abs/1810.04006v3>.
- [3] Andrew V. Goldberg and Alexander V. Karzanov. Path problems in skew-symmetric graphs. *Combinatorica*, 16(3):353–382, 1996. doi: 10.1007/BF01261321. URL <https://doi.org/10.1007/BF01261321>.
- [4] David S. Johnson, Mihalis Yannakakis, and Christos H. Papadimitriou. On generating all maximal independent sets. *Information Processing Letters*, 27(3):119–123, 1988. doi: 10.1016/0020-0190(88)90065-8. URL [https://doi.org/10.1016/0020-0190\(88\)90065-8](https://doi.org/10.1016/0020-0190(88)90065-8).
- [5] Oliver Kullmann and Edward Clewer. Simple minimally unsatisfiable subsets of 2-CNFs. *arXiv preprint arXiv:2603.10944*, March 2026. doi: 10.48550/arXiv.2603.10944. URL <https://arxiv.org/abs/2603.10944v1>. Version 1, submitted March 11, 2026.
- [6] Mark H. Liffiton and Karem A. Sakallah. Algorithms for computing minimal unsatisfiable subsets of constraints. *Journal of Automated Reasoning*, 40(1):1–33, 2008. doi: 10.1007/s10817-007-9084-z. URL <https://doi.org/10.1007/s10817-007-9084-z>.
- [7] R. C. Read and R. E. Tarjan. Bounds on backtrack algorithms for listing cycles, paths, and spanning trees. *Networks*, 5(3):237–252, 1975. doi: 10.1002/net.1975.5.3.237. URL <https://doi.org/10.1002/net.1975.5.3.237>.